

INSTITUTO FEDERAL DE SANTA CATARINA

TRABALHO FINAL

Sistema Supervisório para Quadro de Transferência Automática (QTA)

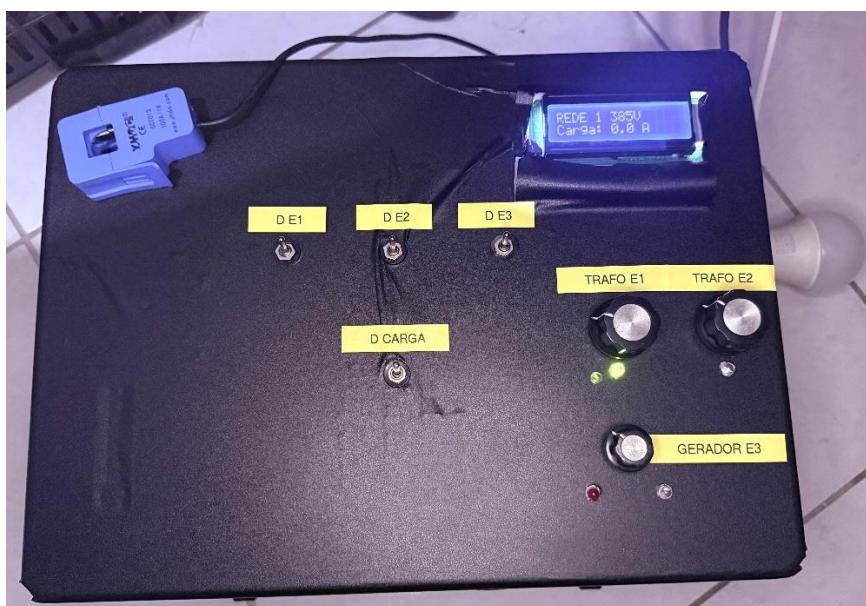


Figura 1 — Vista geral do protótipo do sistema QTA montado.

Aluno(a):	Renato Teodoro
Matrícula:	20261400162
Curso:	ESPECIALIZAÇÃO EM COMPUTAÇÃO CIENTÍFICA PARA A INDÚSTRIA
Disciplina:	AQUISIÇÃO, TRATAMENTO E VISUALIZAÇÃO DE DADOS
Professores:	Fernando Lila , Maurício Stivanello, Delcino Júnior
Turma:	2026 .1
Período de entrega:	14 a 21 de maio de 2026

1. Introdução

Este trabalho apresenta o desenvolvimento de um sistema supervisorio web para monitoramento em tempo real de um Quadro de Transferência Automática (QTA) industrial. O QTA é o equipamento responsável por garantir continuidade de energia a cargas críticas, comutando automaticamente entre Rede 1, Rede 2 e Gerador conforme prioridade e condições de tensão detectadas.

O sistema integra três camadas: (1) aquisição de dados, com Arduino Uno e sensores de tensão e corrente; (2) comunicação, com módulo ESP32 via protocolo MQTT e Wi-Fi; e (3) apresentação, com dashboard web em Django, com atualização em tempo real, histórico gráfico e alertas por limiar.

O trabalho atende a todos os requisitos estabelecidos: leitura de grandezas físicas, transmissão via rede, histórico em formato gráfico e emissão de alertas em situação de valor limiar.

2. Descrição do Sistema

A arquitetura é composta por três blocos funcionais interligados: hardware de aquisição (Arduino Uno), gateway de comunicação (ESP32/MQTT) e software supervisorio web (Django). A Figura 2 apresenta o diagrama de blocos geral.

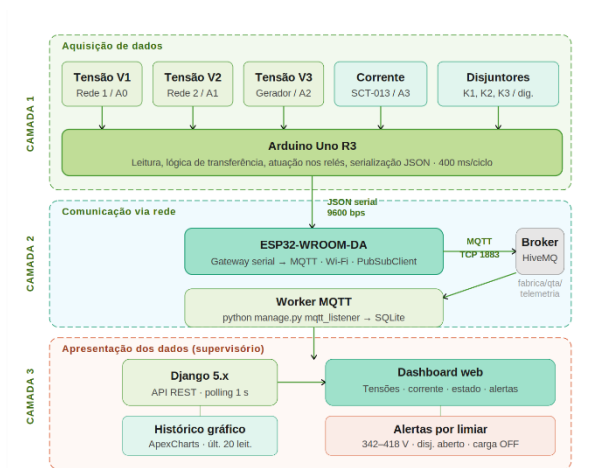


Figura 2 — Diagrama de blocos da arquitetura do sistema QTA Supervisorio.

2.1 Lógica de Transferência em Cascata

O sistema prioriza a Rede 1. Em caso de falha (tensão fora de 342–418 V ou disjuntor aberto), comuta para a Rede 2; se ambas falharem, aciona o gerador. O fechamento de cada contator tem retardo de 5 s para estabilização; a abertura por falha é imediata.

2.2 Variáveis Monitoradas

- Tensão V1 — Rede Elétrica 1 (divisor resistivo/potenciômetro, pino A0)
- Tensão V2 — Rede Elétrica 2 (divisor resistivo/potenciômetro, pino A1)
- Tensão V3 — Gerador (divisor resistivo/potenciômetro, pino A2)
- Corrente de carga — sensor SCT-013 100A/50mA (pino A3)
- Estado dos Contatores K1, K2, K3 e Partida do Gerador (digital)
- Estado do Disjuntor de Carga (digital)
- Contagem de falhas e mensagem de alerta

3. Leitura das Grandezas Físicas

A aquisição é realizada pelo Arduino Uno R3. O firmware, desenvolvido em C++ (Arduino IDE 2.3.8), executa o ciclo de leitura-processamento-atuação-transmissão a cada 400 ms.

3.1 Medição de Tensão

As tensões das três fontes são adequadas à faixa analógica do Arduino (0–5 V) por circuitos divisores de tensão (escala até 500 V). A equação de conversão aplicada é:

$$\text{Tensão [V]} = (\text{leitura_ADC} / 1023) \times 500$$

A janela de operação válida é de 342 V (subtensão) a 418 V (sobretensão), correspondendo a $\pm 10\%$ da tensão nominal trifásica de 380 V.

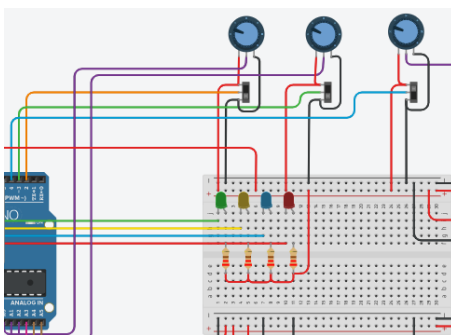


Figura 3 — Circuito divisor de tensão para medição de V1, V2 e V3.

3.2 Medição de Corrente (SCT-013)

A corrente da carga é medida pelo sensor de efeito Hall SCT-013 (100A/50mA), conectado ao pino A3. A biblioteca EmonLib calcula o valor RMS com 1480 amostras por ciclo, garantindo estabilidade na frequência de 60 Hz. Um filtro de ruído descarta leituras abaixo de 0,8 A, eliminando correntes fantasmas por interferência eletromagnética.

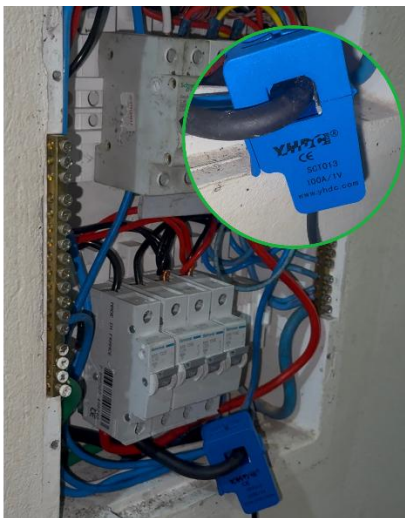


Figura 4 — Sensor de corrente SCT-013 instalado no cabo de alimentação da carga.

3.3 Entradas Digitais e Display LCD

Os estados dos disjuntores (pinos 2–5) são lidos via entradas digitais com lógica invertida: LOW = relé energizado (contator fechado), HIGH = relé desenergizado. Um display LCD 16x2 I2C (endereço 0x27) exibe localmente a fonte ativa, tensão e corrente, alternando a cada 2 s com a lista de falhas ativas.



Figura 5 — Display LCD 16x2 exibindo fonte ativa e corrente de carga.

4.3 Gateway ESP32

O firmware do ESP32 usa as bibliotecas WiFi.h e PubSubClient.h. Ele aguarda uma linha JSON na Serial2 (RX2=16, TX2=17) e ao recebê-la publica imediatamente no broker, com reconexão automática em caso de queda de enlace.

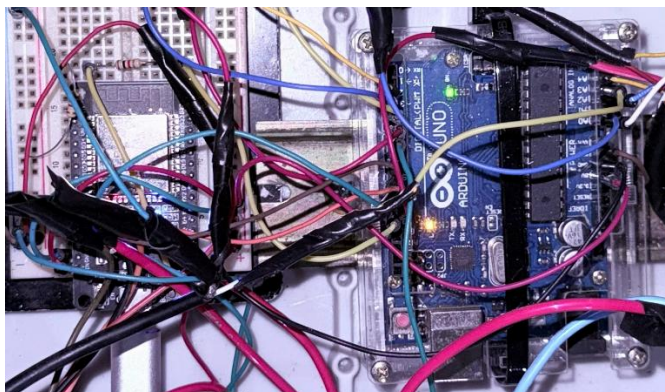


Figura 7 — Módulo ESP32-WROOM-DA conectado ao Arduino Uno por comunicação serial.

5. Sistema Supervisório (Dashboard Web)

O dashboard foi desenvolvido em Django 5.x, acessível em <http://localhost:8000>. O backend persiste cada leitura no banco SQLite via modelo LeituraQTA; o frontend atualiza os dados via polling a cada 1 s na API REST.

5.1 Arquitetura do Software

A aplicação é executada em dois processos paralelos:

- Worker MQTT (python manage.py mqtt_listener): inscreve-se no tópico MQTT e salva cada mensagem como registro no banco de dados.
- Servidor Web Django (python manage.py runserver): serve o dashboard HTML e expõe os endpoints da API REST para o frontend.

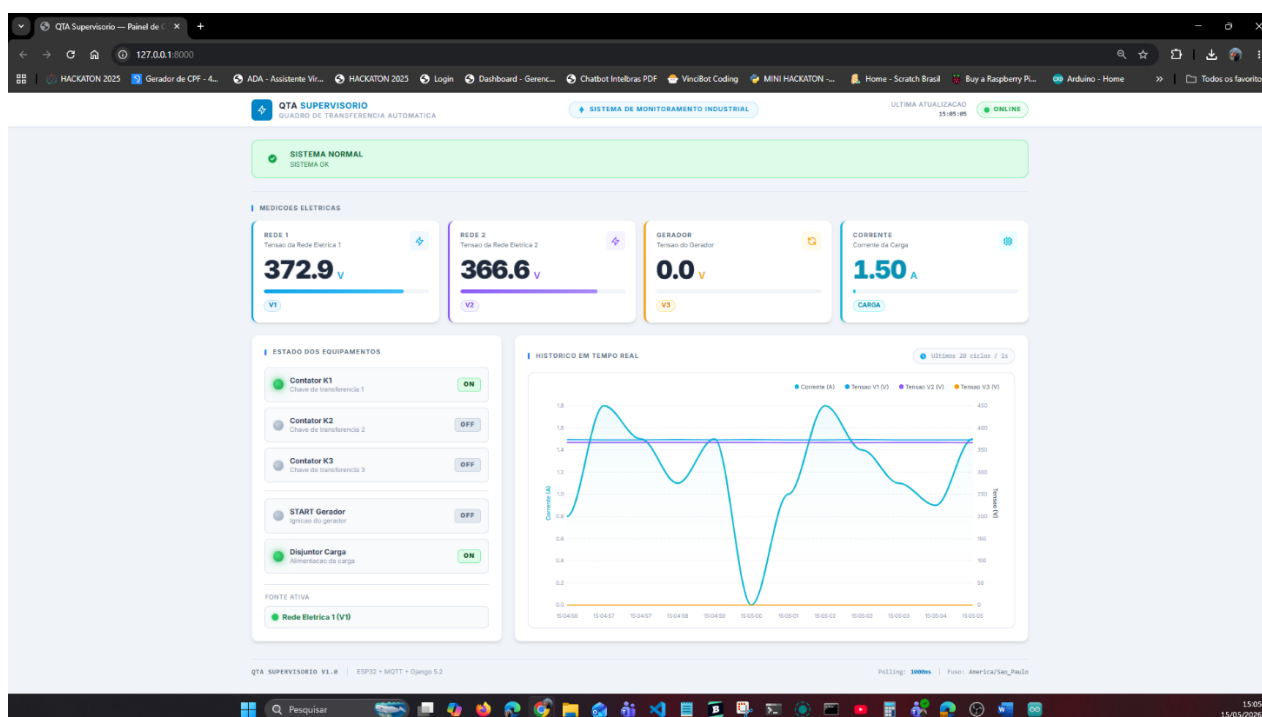


Figura 8 — Dashboard web QTA Supervisório com estado em tempo real do sistema.

5.2 Grandezas Exibidas em Tempo Real

- Cards de Medição: tensões V1, V2, V3 e corrente de carga, atualizados a cada segundo.
- Estado dos Equipamentos: estado (ON/OFF) de K1, K2, K3, partida do gerador e disjuntor de carga.
- Banner de Status: verde 'SISTEMA NORMAL' sem falhas, ou vermelho com descrição da(s) falha(s) ativa(s).

5.3 Histórico em Formato Gráfico

O gráfico de série temporal (ApexCharts) exibe as últimas 20 leituras com curvas distintas para corrente (A) e tensões V1, V2, V3 (V), com dois eixos Y independentes para escala adequada de cada grandeza.



Figura 9 — Gráfico histórico em tempo real das últimas 20 leituras.

5.4 Emissão de Alertas por Limiar

As falhas são detectadas no firmware do Arduino e incluídas no campo 'alertas' do payload. No dashboard, sempre que erros > 0, o banner de alerta é exibido com a descrição textual das ocorrências.

Os tipos de alerta contemplados são:

- Subtensão ou sobretensão em qualquer fonte (fora de 342–418 V)
- Disjuntor de rede ou gerador aberto
- Disjuntor de carga desligado
- Falha de tensão no gerador após acionamento da partida

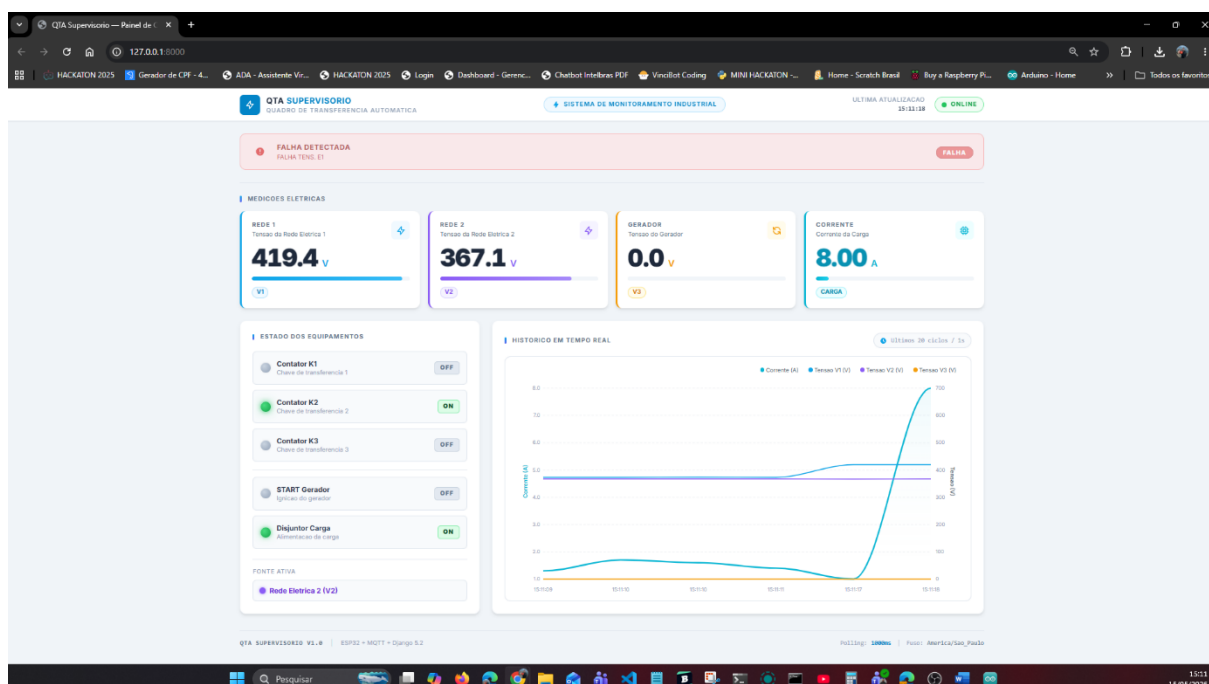


Figura 10 — Dashboard exibindo alerta de falha de tensão ativa.

6. Montagem e Testes

A montagem foi realizada com validação prévia no simulador Tinkercad ([link clique aqui](#)). O diagrama de conexões completo e a montagem física são apresentados nas Figuras 11 e 12.

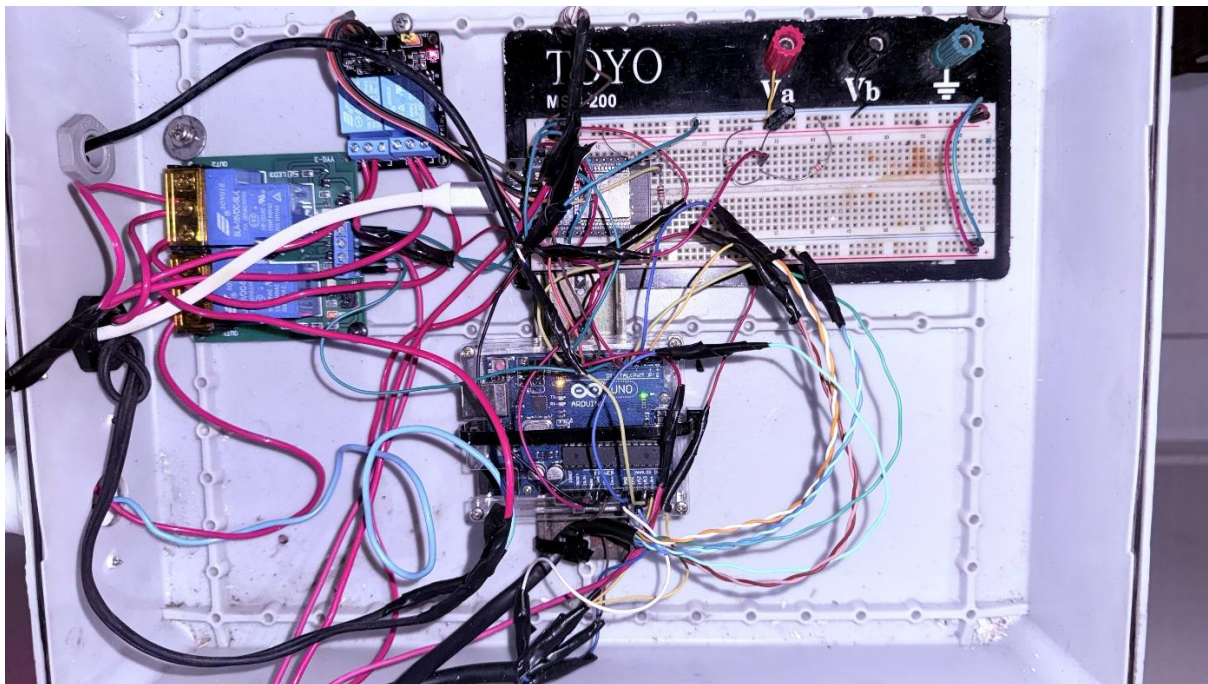


Figura 11 — Montagem física do sistema QTA em bancada de laboratório.

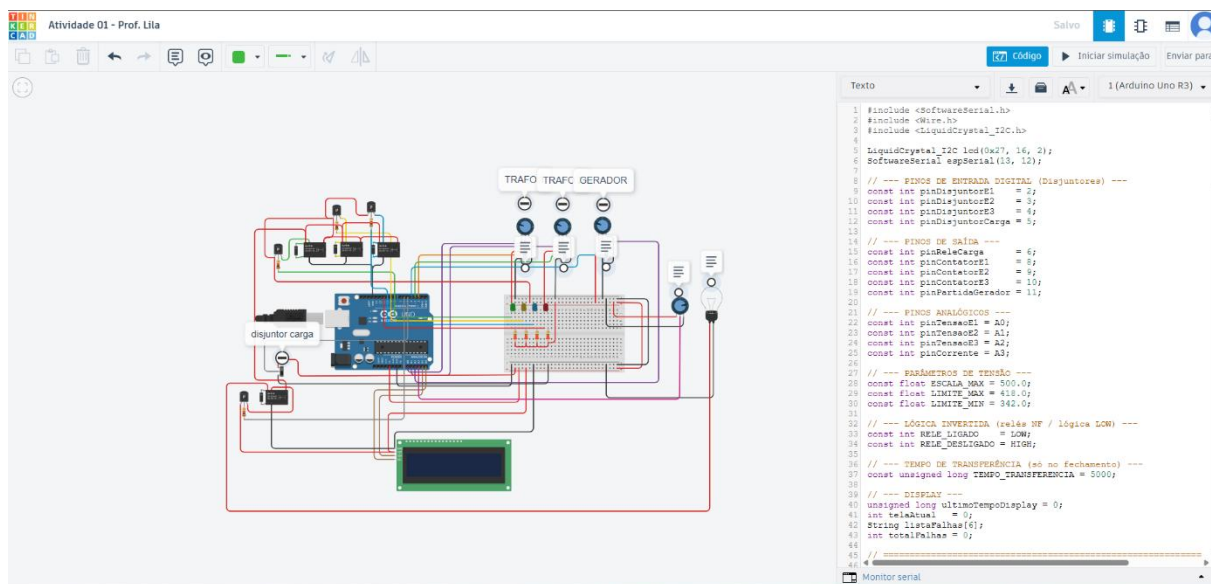


Figura 12 — Simulação do circuito no Tinkercad (Arduino UNO e sensores).

6.1 Procedimento de Testes

1. Validação no Tinkercad: simulação das condições de falha de cada fonte e verificação da lógica de transferência em cascata.
2. Teste de comunicação MQTT: publicação de mensagens via script Python simulador e verificação do recebimento no worker Django.
3. Teste integrado com hardware: verificação da leitura de corrente real com alicate amperímetro e comparação com valores na dashboard.
4. Teste de alertas: simulação de subtensão em Rede 1 e verificação do banner de alerta no dashboard.



Figura 13 — Verificação da leitura de corrente(A) com alicate amperímetro durante os testes integrados.

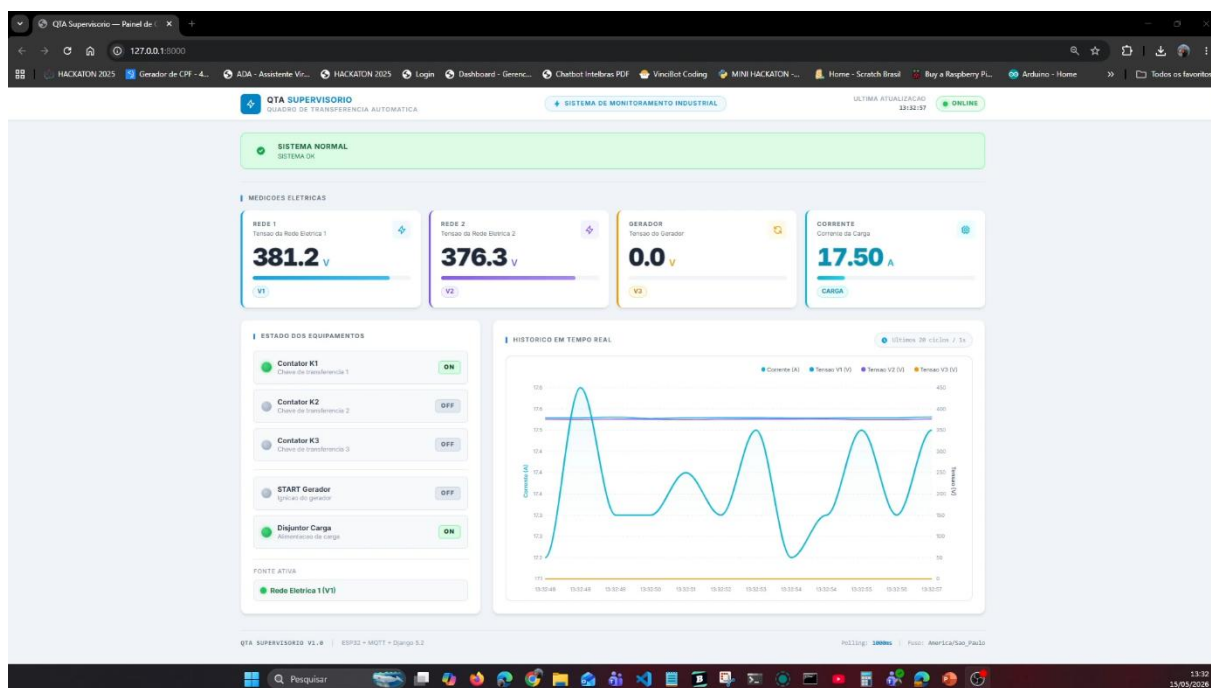


Figura 14 — Comparação da leitura de corrente(A) na dashboard durante os testes integrados.

7. Resultados

O sistema atingiu todos os requisitos propostos. A Tabela 1 consolida os resultados obtidos:

Requisito	Implementação	Status
Leitura de tensão (V1, V2, V3)	Divisor resistivo, escala 500 V, ADC 10 bits	✓ Atendido
Leitura de corrente de carga	SCT-013 + EmonLib (RMS, 1480 amostras)	✓ Atendido
Transmissão de dados via rede	ESP32 + MQTT HiveMQ (Wi-Fi, TCP, 1883)	✓ Atendido
Dashboard web em tempo real	Django 5.x + Tailwind CSS + ApexCharts	✓ Atendido
Histórico em formato gráfico	Gráfico últimas 20 leituras, 4 séries	✓ Atendido
Alertas por limiar	6 tipos de falha detectados e exibidos	✓ Atendido
Lógica de transferência automática	Cascata com retardo 5 s no fechamento	✓ Atendido

Tabela 1 — Consolidação dos resultados obtidos em relação aos requisitos do trabalho.

8. Conclusão

O trabalho demonstrou com sucesso a integração entre hardware de aquisição (Arduino Uno), comunicação sem fio (ESP32/MQTT) e software supervisor web (Django), compondo um sistema SCADA funcional e acessível para monitoramento de Quadro de Transferência Automática.

O protocolo MQTT mostrou-se adequado para o contexto de automação de baixo custo, com baixa latência e consumo reduzido de banda. O dashboard possibilitou acompanhamento em tempo real do estado do sistema, com histórico gráfico claro e alertas de falha precisos.

Como melhorias futuras, identificam-se: uso de broker MQTT privado para maior segurança e disponibilidade, autenticação no dashboard, notificações remotas por e-mail ou SMS em falhas críticas, e armazenamento de histórico estendido em banco de dados dedicado (PostgreSQL) com visualização por período configurável.

9. Referências

ARDUINO. Arduino Uno Rev3 — Documentação e Referência. Disponível em: <https://www.arduino.cc>. Acesso em: maio 2026.

DJANGO SOFTWARE FOUNDATION. Django 5.x Documentation. Disponível em: <https://docs.djangoproject.com>. Acesso em: maio 2026.

ESPRESSIF SYSTEMS. ESP32 Technical Reference Manual. Disponível em: <https://www.espressif.com>. Acesso em: maio 2026.

MQTT.ORG. MQTT Version 3.1.1 — OASIS Standard. Disponível em: <https://mqtt.org/mqtt-specification>. Acesso em: maio 2026.

OPENENERGYMONITOR. EmonLib — Arduino Energy Monitoring Library. Disponível em: <https://github.com/openenergymonitor/EmonLib>. Acesso em: maio 2026.

TINKERCAD. Autodesk Tinkercad — Simulador de Circuitos Online. Disponível em: <https://www.tinkercad.com>. Acesso em: maio 2026.

Link do projeto:

https://www.tinkercad.com/things/csdCUVgv0uG/editel?sharecode=0bJdW3bN1GFG0B-letRhXgJW8FJU45qh-H2ntj4zl_o